KAFKA CHEAT SHEET

1. KAFKA FUNDAMENTALS

1. What is Kafka?

A distributed event-streaming platform for publishing, storing and processing events in real time.

2. Why Kafka?

- High throughput
- Scalable
- Fault tolerant
- Durable and replayable

3. Core Flow



4. Core Components

Component	Meaning
Producer	sends events
Topic	named event stream
Partition	ordered log
Broker	Kafka server
Consumer	reads events
Consumer Group	shares work

5. Example Event

```
key: order-101
value: {"status": "PAID"}
```

→ Kafka decouples producers from consumers.

KAFKA CHEAT SHEET

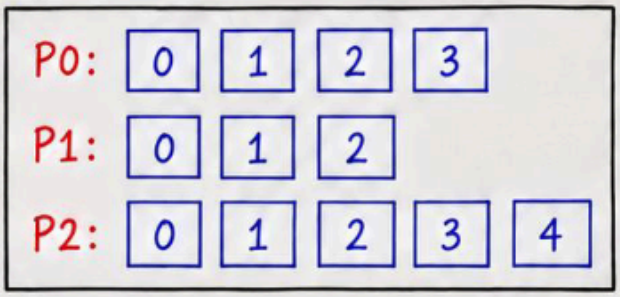
2. TOPICS, PARTITIONS, KEYS & OFFSETS

1. Topic

A named stream of related events.
Example: orders.

2. Partitions

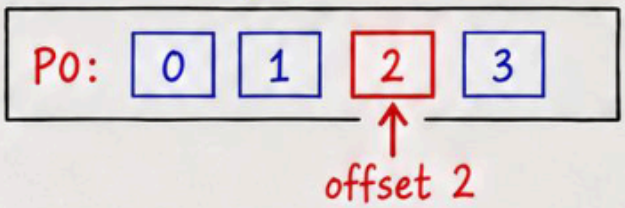
A topic is split into multiple partitions.



Partitions provide parallelism.

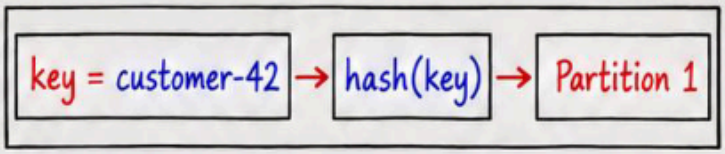
3. Offset

Sequential position of a record inside one partition.



4. Message Key

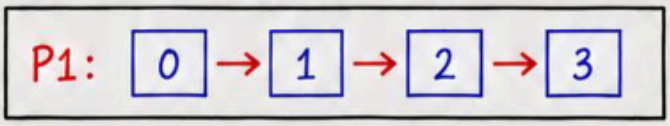
A key is used to decide which partition a record goes to.



The same key normally reaches the same partition.

5. Ordering

Ordering is guaranteed only within one partition.



6. Consumer Parallelism

Multiple consumers can read different partitions in parallel.

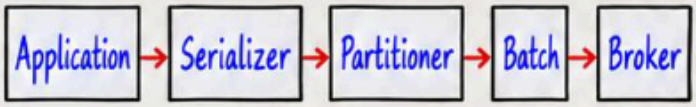
Partitions + Consumers	Result
3 partitions + 2 consumers	Both active
3 partitions + 4 consumers	1 consumer idle

→ Choose the key based on which events must stay ordered.

KAFKA CHEAT SHEET

3. KAFKA PRODUCER

1. Producer Workflow



2. Producer Record

```
ProducerRecord(topic, key, value)
```

Example:

```
new ProducerRecord<>("orders",
    orderId, json)
```

3. acks

acks	Meaning
acks = 0	no confirmation
acks = 1	leader confirms
acks = all	all required ISR confirm

↑ safest

4. Essential Settings

Setting	Meaning
bootstrap.servers	broker addresses
enable.idempotence = true	avoid duplicates
linger.ms	wait for batching
batch.size	batch limit
compression.type	gzip / lz4 / zstd

5. Partition Choice

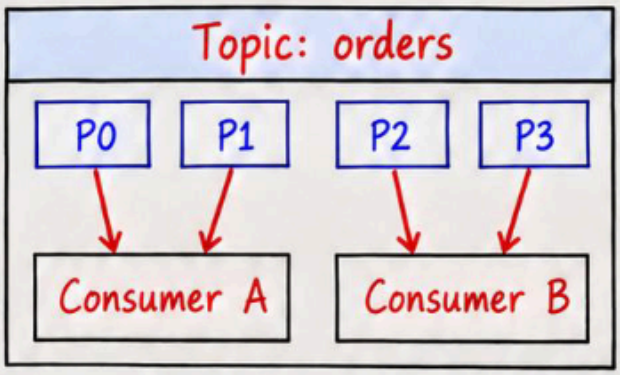
Case	Result
Explicit partition	record goes to specified partition
Key provided	same key always goes to same partition
No key	producer distributes records across partitions

→ Use a stable key, acks=all and idempotence for reliable publishing.

KAFKA CHEAT SHEET

4. CONSUMERS, GROUPS & OFFSETS

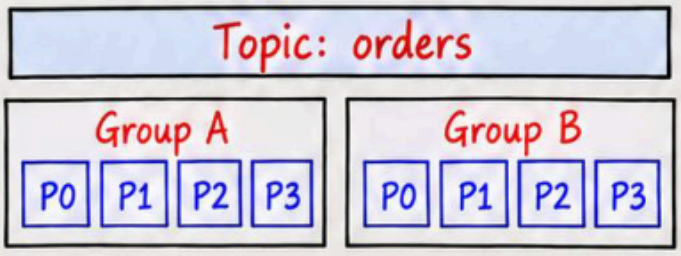
1. Consumer Group



Consumers in one group divide the partitions.

2. Different Groups

Each consumer group receives its own copy of the stream.

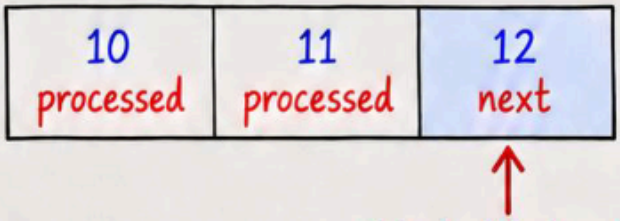


3. Polling

```
consumer.subscribe(List.of("orders"));
records = consumer.poll(Duration.ofMillis(100));
```

4. Committed Offset

The next record the group should read.



5. Commit Strategy

Strategy	Meaming
Auto commit	simple, less control
Manual commit	commit after successful processing
commitSync()	blocking
commitAsync()	non-blocking

6. Rebalancing

Joining, leaving or changing partitions can reassign work.

→ Partitions provide parallelism; committed offsets provide recovery.

KAFKA CHEAT SHEET

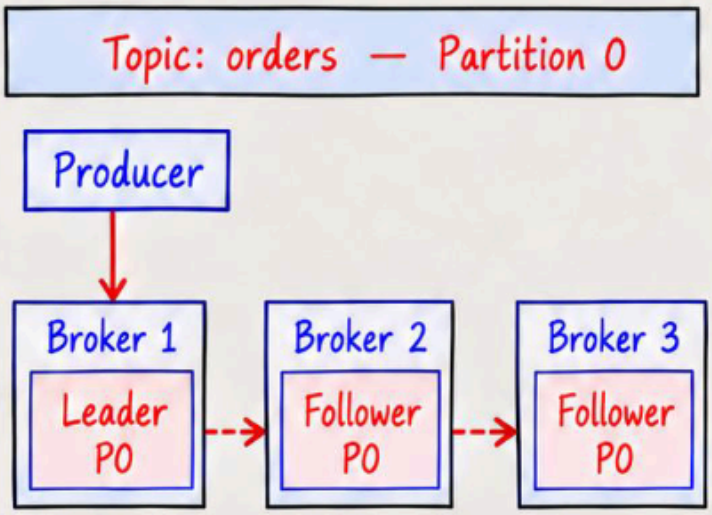
5. BROKERS, REPLICATION & KRAFT

1. Kafka Cluster



Brokers store partitions and serve clients.

2. Partition Replication



3. Leader & ISR

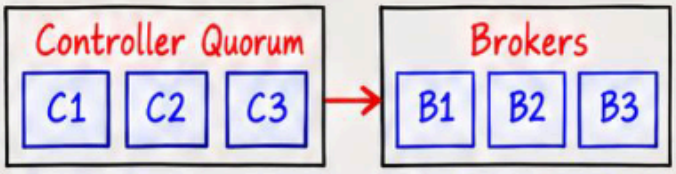
Component	Meaning
Leader	handles reads and writes
Followers	copy the leader log
ISR	replicas caught up with the leader

4. Safe Writes

replication.factor = 3
acks = all
min.insync.replicas = 2

If fewer than 2 ISR remain, writes fail.

5. KRaft



Controllers manage metadata and leader elections.

Modern Kafka uses KRaft; ZooKeeper belongs to older deployments.

→ Replication protects data; ISR determines safe leadership.

KAFKA CHEAT SHEET

6. DELIVERY SEMANTICS & FAILURES

1. Delivery Semantics

Semantic	Meaning
At-most-once	possible loss, no duplicates
At-least-once	no expected loss, duplicates possible
Exactly-once	transactions within Kafka

2. At-Least-Once Flow

Read → Process → Commit

If commit fails, the record may be processed again.

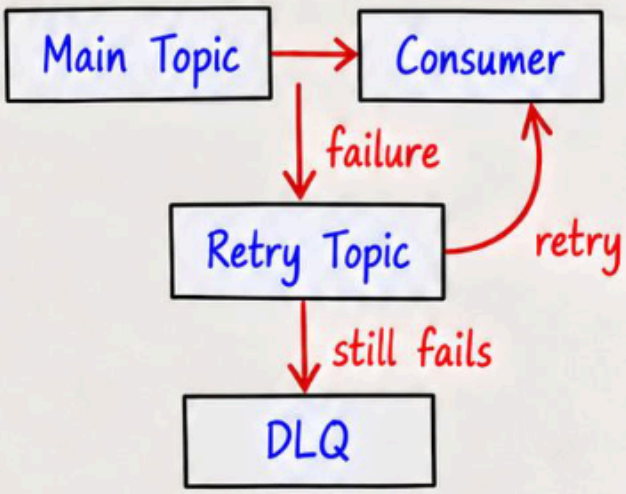
3. Idempotent Processing

Processing the same event twice produces the same final result.

Example:

eventId = order-101
if already processed → skip

4. Retry Pattern



5. Store in DLQ

- original event
- error reason
- topic / partition / offset
- retry count
- timestamp

6. Caution

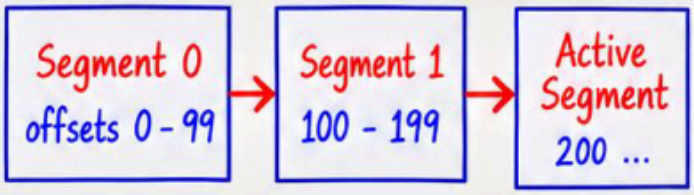
Kafka transactions cover Kafka records. Database or API side effects need idempotency or coordination.

→ Design every consumer for retries, duplicates and replay.

KAFKA CHEAT SHEET

7. RETENTION, COMPACTION & SERIALIZATION

1. Append-Only Log



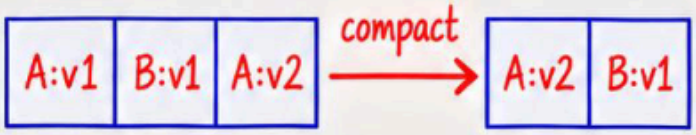
New records are appended to the active segment.

2. Retention

Config	Meaning
cleanup.policy = delete	remove old segments
retention.ms	keep by time
retention.bytes	keep by size

Retention does not depend on whether a consumer has read the event.

3. Log Compaction



Compaction eventually keeps the latest value for each key.

key + null value → delete marker

4. Serialization



5. Common Formats

Format	Meaning
String	simple
JSON	readable
Avro	compact + schema
Protobuf	strongly typed

→ Retention preserves history; compaction preserves the latest state.

KAFKA CHEAT SHEET

8. KAFKA ECOSYSTEM & PRODUCTION

1. Kafka Connect



Connect moves data between Kafka and external systems.

2. Kafka Streams



Streams processes records inside an application.

3. Connect vs Streams

Connect	move data
Streams	process data

4. Security

TLS	encryption
SASL / mTLS	authentication
ACLs	authorization

5. Monitor These

- consumer lag
- under-replicated partitions
- offline partitions
- producer errors and retries
- request latency and disk usage

6. Production Checklist

- ☑ replication factor 3
- ☑ acks=all + suitable min.insync.replicas
- ☑ idempotent producers
- ☑ stable partition keys
- ☑ alerts for lag and replication health
- ☑ capacity test before launch

→ Design for failures and replay—Kafka becomes a reliable event backbone.